Solution Architecture Overview

High-level technical architecture for the MVP, explained for project management

Document	22 - Solution Architecture Overview
Project	AI Customer Support Platform (fictional company: Kestrel Commerce Ltd.)
Prepared by	Project Manager (portfolio author) - self-directed simulation
Status	Simulated baseline v1.0
Project window	5 Jan - 13 Mar 2026 (simulated project window)

SIMULATED PROJECT - Self-directed portfolio case study. Not professional experience, not client work, not employment. All people, figures and outcomes are fictional.

1. Purpose

A deliberately simple architecture for a 10-week MVP, explained for a project manager: what each part does, what it depends on and where the risk sits. The technology choices are proposals made for this simulation; in a real project the engineering team would confirm them.

2. High-level architecture

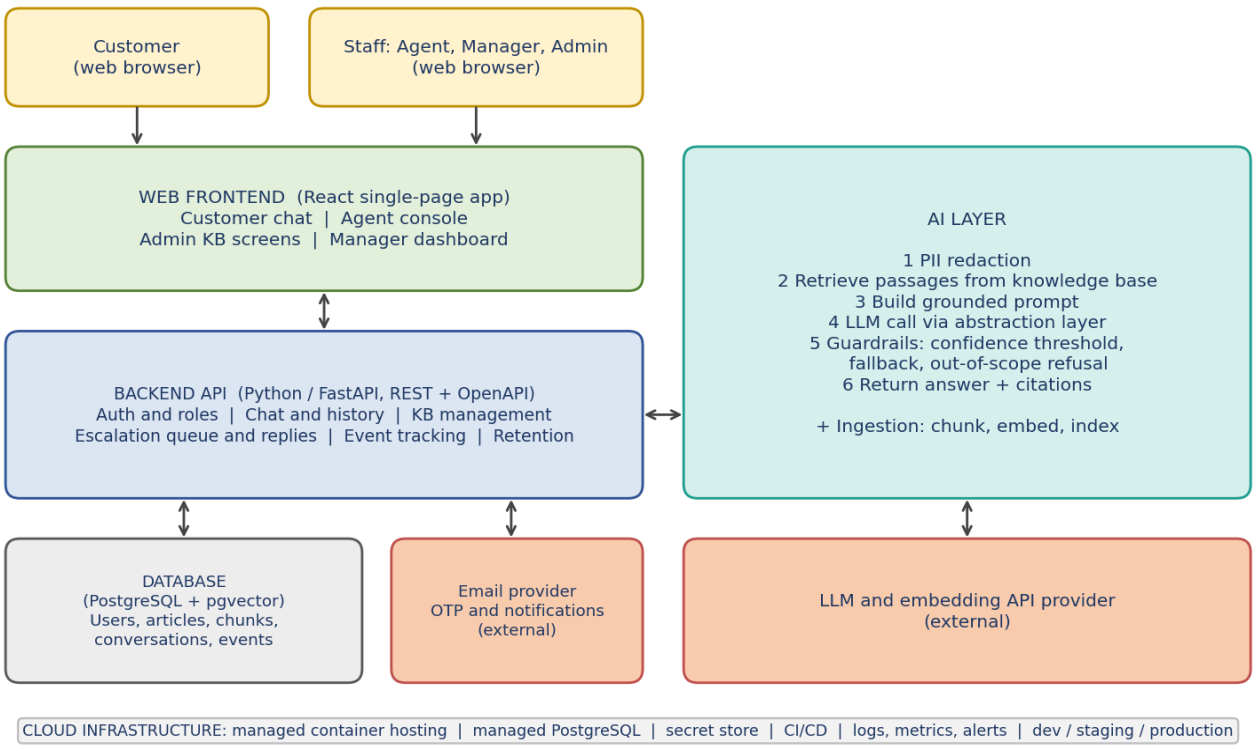


Figure 1: Logical architecture of the MVP.

3. Components in plain language

Component	What it does	Proposed technology	Owner role
Frontend	Everything users see: customer chat, agent console, admin screens, manager dashboard.	React single-page app	Frontend Developer
Backend / API layer	Receives requests, checks who the user is and what they may do, stores and reads data, talks to the AI layer.	Python, FastAPI, REST documented with OpenAPI	Backend Developer
Authentication	Staff sign in with email and password; customers with an emailed one-time code; roles decide access.	Token-based sessions, bcrypt/argon2, email provider	Backend Developer
Database	Stores users, articles, conversations, escalations, events and the search index of article chunks.	PostgreSQL with pgvector extension	Backend Developer
Knowledge base	The approved help articles the AI may use; managed by admins; cleaned before launch.	Tables in the database + admin screens	Product Manager (content), Backend
AI / LLM layer	Finds the most relevant article passages, asks the language model to answer using only those passages, checks confidence and refuses or escalates when unsure; removes personal data first.	Embedding model + LLM via provider API behind an internal abstraction layer	AI/ML Engineer
Analytics	Records usage events (no message text) and turns them into dashboard numbers.	Events table + SQL aggregates	Backend, Frontend
Cloud infrastructure	Runs everything in three environments with automated deployment, monitoring and alerts.	Managed containers, managed PostgreSQL, secret store, CI/CD	DevOps Engineer

4. How one question flows

1. The customer signs in with an emailed code and types a question in the chat window.
2. The backend checks the session and role, then passes the question to the AI layer.
3. Personal data (emails, phone and card patterns) is replaced by placeholders.
4. The AI layer converts the question into an embedding and retrieves the best-matching article passages from the database.
5. If the best match is below the confidence threshold, the assistant returns the fallback and offers a human.
6. Otherwise a grounded prompt is sent to the LLM; the answer is returned with source links and stored with model and prompt versions.
7. The customer can rate the answer or ask for a person; an escalation creates a queue item for agents with the full transcript.
8. Events are logged for the manager dashboard.

5. Technical dependencies

Dependency	Why it matters	Handling
LLM and embedding API	Answers cannot be generated without it; cost, latency and terms affect the project.	Abstraction layer, fallback model validated in spike, spend alerts (R06, R07)
Provider data terms	Real customer data may not be sent until confirmed.	Dependency D-01; test data only until confirmed (R04)
Email provider and DNS	Customer sign-in and notifications depend on email delivery.	Domain verification in Sprint 3; delivery monitoring (R16)
Clean knowledge-base content	Answer quality is capped by content quality.	Content audit (WBS 6.3), content owners (R03)
Cloud account and CI/CD	Every other task needs environments.	Done first in Sprint 1 (WBS 10.1)

6. Technical risks (see Risk Register for scoring)

- Hallucination and wrong retrieval (R01, R02): addressed by grounding, thresholds and a golden-set evaluation gate.
- Privacy and security (R04, R05): redaction, least privilege, rate limits, testing and a Security & Privacy Officer gate.
- Integration (R10) and performance (R11): contract-first APIs, early integration, load test at 50 concurrent sessions.
- Single AI/ML Engineer (R12): pairing with the Backend Developer and written handover notes.

7. Key design decisions

Decision	Rationale	Trade-off
Retrieval-augmented generation instead of fine-tuning a model	Answers come from approved, editable content, can show sources and are easy to audit.	Quality depends on retrieval and content quality.
PostgreSQL with pgvector instead of a separate vector database	One datastore to run, back up and secure; adequate for about 280 articles.	May need revisiting at much larger scale.
Provider abstraction layer	Lets the team switch LLM provider if cost, terms or availability change.	Small extra build effort in Sprint 2.
Single cloud region, three environments	Meets pilot availability target at low cost.	No regional failover in the MVP.
Customer sign-in by email code	No password management and gives conversation history.	Depends on email deliverability.